Containers and Docker

Introduction

Containers and Docker

Agenda

- [08:15 – 08:30] – Setting up Docker on your machines
- [08:30 – 09:00] – Quick intro to Docker
- [09:15 – 12:00] – Follow-along exercises

Preparation

For the exercises:

- Docker: Overview
- Docker: Dockerfile reference
- Docker: Concepts (basics and building images + containers)
- Docker: Get Docker
- Docker: Install Docker Engine (can also be done through Docker Desktop)
- Docker: Introduction

Exercises

The following exercises will guide you through setting up Docker and using it for your data science projects:

- Notes and Theory - Understanding Docker concepts
- Exercise 0: Installation - Install and test Docker
- Exercise 1: Create a Dockerfile - Build Dockerfile for training
- Exercise 2: Build the Image - Build and tag your image
- Exercise 3: Run your Image - Run containers with various options
- Exercise 4: Push and Pull - Work with registries
- Exercise 5: Clean up - Remove unused images

Introduction to Docker

Prelude

Before getting started with using Docker, it's a good idea to motivate and explain *what* Docker actually is so you don't just type commands and see stuff happen.

Why?



To avoid the above from happening. Or even shorter -- reproducibility.

Docker essentially allows running code in an isolated and agnostic way from the operating system, making it crucial for reproducible, automated workflows for both normal DevOps and MLOps.

Itemised list for convenience:

- Reproducible and consistent delivery
- CI/CD pipelines
- Portable and collaborative
- Efficient use of resources

> ***Note:*** *Fun fact: Docker is written in Go.*

Standardization is key for team collaboration. Without it, data scientists and developers waste time debugging environment differences rather than building out the product. Docker solves this by packaging applications and all their dependencies into consistent, portable units.

Next: Technical Details

Technical Details

Technical Details

Docker is a way to create a virtual instance of compute resources. However, unlike virtual machines with their own complete operating system, Docker uses lightweight containers instead.

The key thing is these containers have their own filesystem, dependencies, processes and such, which allows you to run an application in isolation from your operating system.

Containers vs Virtual Machines

Virtual Machines (VMs)

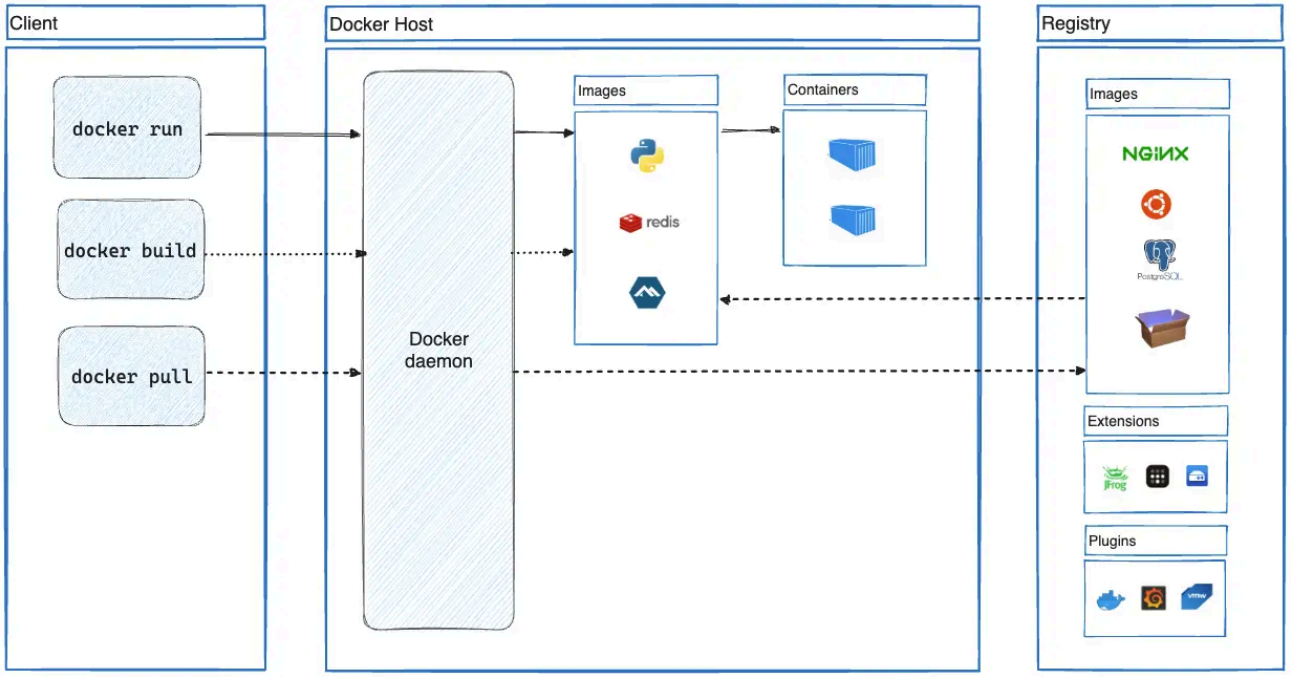
- **Full OS**: Each VM runs a complete operating system with its own kernel
- **Heavyweight**: Requires significant resources (CPU, memory, storage)
- **Slow startup**: Booting a full OS takes time (seconds to minutes)
- **Strong isolation**: Complete separation from host and other VMs
- **Persistent**: Keeps files and processes open

- **Use case**: Running different OS versions or when full isolation is required (e.g., cyber security)

Containers

- **Shared kernel**: Containers share the host OS kernel
- **Lightweight**: Only includes application and its dependencies, not a full OS
- **Fast startup**: Typically starts in milliseconds
- **Process-level isolation**: Applications run in isolated user spaces
- **Ephemeral**: Releases resources after completion
- **Use case**: Running multiple applications on the same OS with resource efficiency (e.g., MLOps)

Docker Architecture Overview



Docker follows a client-server architecture:

- **Docker Daemon (dockerd)**: The background service that manages Docker objects (images, containers, networks, volumes)

- **Docker Client (docker)**: The CLI interface users interact with
- **Docker Registry**: Stores Docker images (Docker Hub is the default public registry)

When you run `docker build`, the client sends the build request to the daemon, which builds the image. When you run `docker run`, the daemon creates and starts a container from the image.

Three Components We Focus On

1. Dockerfiles
2. Docker images
3. Docker containers

You will explore each of these components more closely through the upcoming exercises, but a short introduction is in order.

Dockerfiles

Dockerfiles are a set of instructions that will be used to build a **Docker image**. These instructions include information about the *base image* and the *commands* that will be executed (such as installing packages, copying files, etc).

A typical Dockerfile might look like:

```
FROM python:3.14-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

- `FROM`: Specifies the base image
- `WORKDIR`: Sets the working directory in the container

- `COPY`: Copies files from the build context to the container
- `RUN`: Executes commands during the build process
- `CMD`: Specifies the command to run when the container starts

Docker Images

A **Docker image** is the output from building the **Dockerfile**. They are:

- **Immutable**: Once built, images cannot be changed (any changes create a new image)
- **Layered**: Each instruction in the Dockerfile creates a layer; these layers are stacked to form the final image
- **Cached**: Each layer is cached, enabling fast rebuilds when only some parts change
- **Versioned**: Images can be tagged and versioned (e.g., `myapp:v1.0`, `myapp:latest`)

These images can be pushed to and pulled from image repositories. Docker Hub is the default public registry, but organizations often use private registries (AWS ECR, Google Container Registry, etc.).

The layered structure means that when you rebuild an image after changing only one line in your application code, Docker will reuse all the cached layers up to the point where the change occurs, resulting in fast and efficient build times.

Docker Containers

When you want to run the **Docker image**, it spins it up in a **Docker container** as an isolated runtime environment. The container lifecycle is:

1. **Create**: `docker create` creates a container from an image but doesn't start it
2. **Start/Run**: `docker run` creates and starts a container in one command
3. **Execute**: The container runs the specified command
4. **Stop**: `docker stop` gracefully stops a running container
5. **Exit**: The container shuts down and releases compute resources

6. **Remove**: `docker rm` removes the container (storage is released)

Although they are based on the immutable **Docker image**, it is possible to customize the **Docker container** at runtime. This can be done by:

- Overriding the default command
- Mounting volumes for persistent storage
- Passing environment variables
- Publishing ports

This approach also ensures the same environment for *development* and *production* work, eliminating the "works on my machine" problem.

Next: Industry Usage

Docker in Practice

Docker in Practice

Docker has become a standard tool in the software industry for packaging and deploying applications. Its adoption spans from small startups to large enterprises, particularly in DevOps and MLOps workflows.

Common Use Cases in Industry

Development Environment Standardization

One of the most widespread uses of Docker is standardizing development environments across teams. This is especially valuable in data science teams where projects often have complex dependency chains (specific versions of Python, ML libraries, system libraries).

Microservices Architecture

In modern software development, applications are increasingly built as collections of microservices rather than monolithic applications.

- **One service per container**: Each microservice runs in its own container
- **Independent scaling**: Services can be scaled independently based on demand
- **Technology diversity**: Different services can use different technology stacks
- **Isolated failures**: A failure in one service doesn't crash the entire application

At Maersk you might for example see Docker being used multiple ways:

- Data pipelines with tenant-specific images built on top of main image
- Internal MLOps orchestrator
- AI/ML (sub)projects written in either Python or R (or both)
- Various internal applications built by different teams

ML Model Serving and Training Environments

In machine learning workflows, Docker plays several crucial roles:

Training Environments:

- Pre-configured containers with all necessary dependencies (Python, CUDA, ML frameworks)
- Reproducible training across different hardware configurations
- Easy sharing of training environments between researchers
- Version control of training environments alongside model code

Model Serving:

- Model inference APIs packaged as containers
- Consistent environment from training to production
- Easy deployment across different serving platforms (cloud, on-premise, edge)
- Quick rollback to previous model versions

CI/CD Pipelines

Docker is a fundamental building block for Continuous Integration and Continuous Deployment pipelines:

- **Build stage**: Code is compiled and dependencies are installed in a container
- **Test stage**: Automated tests run in isolated containers
- **Deploy stage**: Application containers are deployed to staging/production

Benefits:

- Consistent build environments across all pipeline stages
- Parallel execution of tests in separate containers
- Easy cleanup of build artifacts after pipeline completion
- Reproducible builds that can be traced and debugged

Orchestration

When you have many different containers working together, you can orchestrate them in different ways. It is not something that we will be focusing much on here, but it gives you some context to what you might encounter in some form or another.

A simple approach *Docker Compose* is for manual setup of multiple containers. It's useful for development work, and it is a single YAML configuration file that essentially has instructions on how to run multiple containers at once.

A more complex approach is *Kubernetes*, which is for a more consistently deployed service. It automates deployment, scaling and management of containerized applications. This means you have a cluster of machines that runs the pods that hold the containers. It uses more advanced health checks, load balancing storage orchestration etc, which is for example useful for when web sites need to serve higher demands.

Image Repositories

Docker Hub

Docker Hub is Docker's default public registry:

- Hosts official images for popular software (Python, Node.js, PostgreSQL, etc.)
- Allows users to share their own images publicly
- Provides build automation (automatically build images when source code changes)

- Offers both free and paid tiers

Private Registries

For enterprise use, private registries provide:

- Control over who can access images
- Compliance with security and data privacy requirements
- Integration with existing authentication systems
- On-premise or cloud-based hosting

Popular private registry options:

- **AWS Elastic Container Registry (ECR)**: Managed Docker registry on AWS
- **Google Container Registry (GCR)**: Google Cloud's registry service
- **Azure Container Registry (ACR)**: Microsoft's offering on Azure
- **GitHub Container Registry**: Integrated with GitHub repositories
- **Self-hosted**: Options like Docker Registry, Harbor, or Nexus

---

Versioning and Rollback Strategies

You can tag your images so you can easily find the exact image you want/need.

Tagging strategies:

- **Semantic versioning**: `v1.0.0`, `v1.1.0` for release versions
- **Git SHA**: `git-abc1234` for traceability to source code
- **Timestamp**: `2025-09-27` for build date tracking
- **Environment**: `dev`, `staging`, `prod` for different environments
- **Latest**: Floating tag for the most recent build (use with caution)

Best Practices for Production

1. **Start with the slimmest image** and choose the right image for your use-case
2. **Handle secrets properly** (e.g., API credentials) - never hardcode them in images, but read them from environment variables which can be specified at runtime
3. **Decouple applications** by focusing on a single concern per container
4. **Pin base image versions** but update them often to get security fixes
5. **Test with CI/CD pipelines** to ensure your containers work correctly

Next: Docker Alternatives

Docker Alternatives

Docker Alternatives

While Docker is the most popular containerization platform, they have a licensing setup so it's only free for: personal use, education, non-commercial open source projects amd small businesses (fewer than 250 employees OR less than $10 million in annual revenue).

Luckily several alternatives exist for multiple different use cases and requirements, so for the sake of completeness, here's a list of some for the interested people. Don't spend too much time on this, and consider doing your own research into each if you want to look more into FOSS products.

Podman

Podman (Pod Manager) is the most direct alternative to Docker and is often directly interchangeable.

Features

- **Drop-in replacement**: Podman's CLI is designed to be compatible with Docker's CLI. Most `docker` commands work with `podman` with minimal changes (`alias docker=podman` is a common setup).
- **Daemonless**: Unlike Docker, which requires a background daemon (dockerd), Podman runs containers directly through fork/exec in user space. There is no root-owned daemon to attack.
- **Docker compatibility**: Pulls images from Docker Hub, builds from Dockerfiles, and works with Docker Compose (via `podman-compose` or the compose provider in newer versions).
- **Pod support**: Manages groups of containers as pods, mirroring the Kubernetes pod concept, which eases later migration to Kubernetes.

Differences

- **Security model**: Rootless by default for regular users, using user namespaces. No setuid binaries. Docker historically ran everything through a root daemon, which is a larger attack surface.
- **Systemd integration**: `podman generate systemd` (and Quadlet in newer versions) produces systemd units for service management; Docker relies on its own daemon restart policies.
- **No daemon means no single point of failure**: A crashing build does not take down other running containers. The trade-off is weaker support for some Docker tooling that expects a live daemon API (Podman exposes a compatibility socket to mitigate this).

nerdctl

nerdctl (contaiNERD CTL) is a Docker-compatible CLI for containerd, the runtime that Kubernetes itself uses. GitHub

Features

- **Docker-compatible UX**: Same command feel as `docker` and `podman`; CLI syntax follows Podman conventions. Medium (author)
- **Compose support**: `nerdctl compose up` for multi-container stacks. dev.to
- **Rootless and UserNS-Remap modes**: Runs containers fully unprivileged, or remaps UIDs under a root containerd. dev.to
- **Cutting-edge containerd features**: Lazy pulling (eStargz, Nydus, OverlayBD, SOCI), image encryption via ocicrypt, and IPFS distribution. GitHub

Differences

- **Goal**: Not competing with Docker. It exists to expose containerd features Docker has not adopted yet, and is useful for debugging Kubernetes nodes since Kubernetes containers live in containerd. Medium
- **Requirements**: Needs containerd, CNI plugins for `nerdctl run`, and a separate BuildKit daemon for `nerdctl build`. Docker ships all of this as one bundle. GitHub
- **Flag coverage**: Not every Docker CLI flag is implemented; some unimplemented flags reflect containerd design, not missing functionality. GitHub docs

Apptainer (formerly Singularity)

Apptainer is the dominant container system for HPC (high-performance computing), designed for multi-tenant clusters with untrusted users. CIQ

Features

- **Single-file SIF images**: Containers are packaged as one portable, verifiable file, easy to share across HPC environments. CIQ
- **Rootless since inception**: Built for unprivileged users in shared environments; supports user-namespace mode without setuid. Rootless Containers
- **Signing and encryption**: Cryptographic signing of images, plus native MPI support and Slurm/PBS scheduler integration. CIQ
- **Runs Docker images**: Can execute Docker/OCI images directly, so existing images stay usable. RUG HPC docs

Differences

- **Application-level, not system-level**: User IDs inside are mapped to a single user; it cannot run many system-level applications and does not fully meet OCI runtime requirements. Rootless Containers
- **HPC-centric model**: Runs jobs as the invoking user, integrates with batch schedulers, no long-lived daemon or service model. Docker's daemon-and-service model is a poor fit for shared clusters where no user is trusted. Apptainer docs
- **Fork history**: The original project moved to the Linux Foundation and was renamed Apptainer; a pre-fork open-source version continues as SingularityCE with setuid as default. Rootless Containers Note that CIQ's page is a vendor site, so treat its marketing framing accordingly.

LXC

LXC (Linux Containers) is the older OS-level virtualization project that predates Docker and produces system containers rather than application containers. Docker blog

Features

- **System containers**: Runs a full OS userspace including systemd/PID 1, more like a lightweight VM than a single-process container. Ars Technica forum
- **Kernel-native isolation**: Uses namespaces and cgroups directly for isolation and per-container resource management. Docker blog
- **Persistent, long-lived environments**: Designed for hosts that run multiple services and need direct OS control, rather than ephemeral app packaging.

Differences

- **No image-centric workflow**: Docker's core value is packaging, shipping, and running applications as immutable images; LXC is oriented toward long-running machine-like environments. CyberPanel guide
- **Different audience**: LXC suits admins wanting a lightweight VM replacement with OS control; Docker suits developers deploying apps consistently. Docker blog
- **Weaker portability story**: No equivalent of Docker Hub image distribution as a first-class concept; migration is between directories and snapshots rather than images. Note that the two comparison sources here are blog/vendor content, not primary documentation.

Kaniko

Kaniko is not a runtime alternative but a build-tool alternative: it builds and pushes container images from Dockerfiles without any Docker daemon. kaniko.org

Features

- **Daemonless builds**: Executes each Dockerfile command in userspace and snapshots the filesystem, then pushes the result to a registry; no Docker daemon or CLI involved. Google Cloud Blog
- **Unprivileged**: Runs in standard Kubernetes pods without privileged mode or root, avoiding the Docker-in-Docker security problem in CI/CD. OneUptime
- **Dockerfile compatible**: Supports standard Dockerfile workflows, multi-stage builds, layer caching, and registry credentials via Kubernetes secrets. OneUptime

Differences

- **Build only**: Cannot run containers. You still need Docker, Podman, or a Kubernetes runtime to execute what it builds.
- **Kubernetes-native context**: Meant to run as a pod with the build context in a volume or bucket; it is not a laptop dev tool.
- **Project status**: Google archived the original repo in June 2025; it continues via the Chainguard fork maintained by the original creators. Mention this when choosing it for long-term CI infrastructure. ComputingForGeeks

Next: Exercise 0 - Installation

Installation

Exercise 0: Installation

As described in the previous section, Docker is really a way to run code in a reproducible way.

Exercise 0: Installation

1. ### Install Docker Desktop+Engine

   If you haven't already, you need to make sure to install Docker Engine. The easiest way is to do it through Docker Desktop. For Windows users, you will also need to install WSL if you haven't already.

2. ### Test your Docker installation is working

   In the terminal, run `docker run hello-world`

   If it works, great! If not, try googling the issue.

   A common issue is that the Docker daemon not running. If you installed it with Docker Desktop, make sure that program is running.

Next: Exercise 1 - Create a Dockerfile

Create a Dockerfile

Exercise 1: Create a training Dockerfile

> ***Note — Learning outcomes*** *By the end of the exercises, we expect you to be able to do the following:*

> - *Write/create a basic Dockerfile that can run a basic DS project*
> - *Explain the workflow and processes for using Docker*
> - *Motivate the typical ways why you would modify Docker containers at runtime*
> - *Describe image repositories and explain their usage*

1. ## Set up a project folder with the training script

   Create a new project directory and copy in the training script from `resources/train.py`:

   ```
   mkdir my-docker-project
   cp resources/train.py my-docker-project/
   cd my-docker-project
   ```

   The script trains a LogisticRegression classifier on the Iris dataset and saves the model to `artifacts/model.pkl`. You will also need a `requirements.txt` listing the packages it depends on.

2. ## Create train.dockerfile

   You can use your favourite tool to do this. Or do:

   ```
   touch path/to/project/train.dockerfile
   ```

3. ## Find appropiate base-image

   A `python:3.x` image, for example `python:3.11`.

4. ## Modify example Dockerfile to fit our use-case

   Inspired from this website:

```
FROM python:3.11
WORKDIR /usr/local/app
# Install the application dependencies
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
# Copy in the source code
COPY train.py ./
# Setup an app user so the container doesn't run as the root user
RUN useradd app
USER app
CMD ["python", "train.py"]
```

Next: Exercise 2 - Build the Image

Build the Image

Exercise 2: Build the image

1. **Build the train.dockerfile**

   Normally you just need to run `docker build .`, but since we want to use a specific .dockerfile, we have to modify it slightly:

   Hint:

   `docker build -f train.dockerfile .`

2. **Add a meaningful name by version/tag**

   Just building the image can make it hard to find again.

Adding tags when building the image is typically the easiest way to deal with this. You can also add tags after it has been built but then you need to know the image id.

Hint:

```
docker build -f train.dockerfile -t iris-train:1.0.0 .
```

3. ### Changing files/fixing errors

   It's possible that you need to rebuild the image because of some errors. Can you spot what the error is now?

   ### Hint:

   Python version and requirements.txt does not have scikit-learn

   ```
   Question: why is it faster building it now?
   ```

Next: Exercise 3 - Run your Image

Run your Image

Exercise 3: Run your image

1. ### Run your latest image

   The basic `docker run <image>` is pretty straight-forward. But what is your

image id?

Hint to find list of images

`docker images`

2. ## Solving issues with different arguments

   Full list of arguments here

   -i (--interactive) for going "into" the container and run commands interactively.

   -d (--detach) to don't have the process run in your terminal but detached in the background instead.

   -p (--publish)

   `docker run -p 10000:8080 iris-train`

   -v (--volume) for mounting a directory, allowing you to access files there outside the container.

   `docker run -v ./artifacts:/usr/local/app/artifacts iris-train`

   -e (--env) is for setting environment variables, which can sometimes be useful whenever using env variables in your code.

3. ## Check running containers

   Checking running containers can be done by `docker ps`. What about stopped containers?

   `docker ps -a`

Next: Exercise 4 - Push and Pull

Push and Pull

Exercise 4: Push and pull

1. ## Find an interesting AI/ML image and pull it

   `docker pull ghcr.io/mlflow/mlflow`

   And tag it: `docker tag ghcr.io/mlflow/mlflow mlflow:latest`

   Or run it directly: `docker run ghcr.io/mlflow/mlflow`

   ```
   Question: Suggested arguments for running this image?
   ```

   ### My suggestions:

   `docker run -p 5005:5000 -v $(pwd)/mlruns:/mlruns -e MLFLOW_ALLOW_FILE_STORE=true -d mlflow:latest mlflow server --host 0.0.0.0 --backend-store-uri /mlruns`

2. ## Pushing your image

   This will require setting up a user that can push to whichever registry you want to push it to. Generally it would follow this structure:

   ```
   docker login <REGISTRY_HOST>:<REGISTRY_PORT>
   docker tag <IMAGE_ID> <REGISTRY_HOST>:<REGISTRY_PORT>/<APPNAME>:<APPVERSION>
   docker push <REGISTRY_HOST>:<REGISTRY_PORT>/<APPNAME>:<APPVERSION>
   ```

   An example would look like:

   ```
   docker login repo.company.com:3456
   docker tag 19fcc4aa71ba repo.company.com:3456/myapp:0.1
   docker push repo.company.com:3456/myapp:0.1
   ```

   Or in our case with default Docker Hub:

```
docker login --username jeppetk
docker tag 90c46295c455 jeppetk/iris-train:initial
docker push jeppetk/iris-train
```

3. ### What do the hashes represent when pushing/pulling?

   It's the different layers. This can be confirmed with the command `docker inspect 90c46295c455`

Next: Exercise 5 - Clean up

Clean up

Exercise 5: Clean up

1. ### Check local images

   `docker images`

   What about images that are not tagged or only exist as intermediate layers?

   #### Show all images

   `docker images -a`

How much space are your images taking up in total?

Show the total size of your images

`docker system df` gives you an overview of how much disk space Docker is using, including the total size of all your images. `docker images` also shows the size of each individual image.

2. ## Remove images

   Try removing one of your images by name:

   `docker rmi iris-train:latest`

   ```
   Question: The image is still listed by docker images. Why?
   ```

   ### Answer

   Removing an image by *name* only removes that *tag*. If the same image has other tags (like `latest` and the registry name `jeppetk/iris-train:initial`), the image itself is still referenced by the remaining tags and stays on disk.

   Now try removing it by *image id* instead:

   `docker rmi 90c46295c455`

   ### What happens?

   Removing by image id deletes the image completely (all tags pointing to it are removed), as long as nothing else references it.

   Did you get an error like `image is being used by stopped container`?

   ### The common issue: images attached to containers

   You cannot remove an image that a container is based on. This goes for **running** containers too: if you try `docker rmi` on an image that a running container uses, Docker refuses with `image is being used by running container` — and even `docker rmi -f` cannot fully delete it while the

container is running. You need to stop the container first, remove it (see step 3), and then remove the image. `docker rmi -f` does force removal past *stopped* containers — use with care.

3. ## Remove containers

   Stopped containers pile up quickly. Remove one by name or id:

   ```
   docker rm <CONTAINER_ID>
   ```

   Or all stopped containers at once:

   ```
   docker container prune
   ```

   ```
   Question: What happens to the data the container wrote to a mounted vo
   ```

   ### Answer

   Nothing! Removing a container does **not** delete your data. Anything written to a volume or bind mount (like `-v ./artifacts:/usr/local/app/artifacts`) still lives on your computer, and you can access it again from a new container mounting the same volume.

   The `-v` flag on `docker rm` exists for the opposite reason: `docker rm -v <CONTAINER_ID>` removes the *anonymous volumes* that Docker created for the container. Named volumes and bind mounts are never touched, so your mounted data stays on your computer either way.

4. ## Prune images and containers

   Docker offers a broom for sweeping everything unused away in one go:

   ```
   docker image prune
   ```

   ```
   docker container prune
   ```

You may have seen the word *dangling* in the output. What does it mean?

What dangling images are

A dangling image is an image layer that no longer has a tag and is not referenced by any tagged image. This typically happens when you rebuild an image with the same tag (like `latest`): the old image loses its tag and becomes `<none>:<none>`. They show up in `docker images` as `<none>` and are safe candidates for cleanup.

Warning: pruning cannot be undone. Usually that is fine — you can build your own images again from the Dockerfile, and pull the rest down again. But an image with no Dockerfile behind it is gone for good. `docker image prune -a` removes *all* unused images, not just dangling ones, so think twice before adding `-a`.

Back to Introduction